```
InsightPulse/
 |
 ├── README.md
 ├── ACCURACY.md            # your documented test results (deliverable #4)
 |
 ├── data/
 |    ├── raw/              # original feedback dataset, untouched
 |    ├── processed/        # cleaned dataset
 |    └── reshape_data.py   # cleaning/formatting script
 |
 ├── backend/
 |    ├── requirements.txt
 |    ├── main.py           # starts FastAPI, lists routes
 |    ├── models.py         # pydantic schemas for feedback + classification output
 |    ├── db.py             # PostgreSQL connection, table setup
 |    ├── taxonomy.py       # your fixed category list
 |    ├── prompts.py        # every prompt + few-shot examples, in one place
 |    ├── pipeline.py       # classify → validate → store logic
 |    ├── aggregator.py     # weekly grouping, theme counts, sentiment trend
 |    ├── summary.py        # generates the AI narrative weekly summary
 |    ├── embeddings.py     # text → vector, via Sentence-Transformers
 |    ├── vector_store.py   # save/search embeddings via pgvector in PostgreSQL
 |    ├── chatbot.py        # routes question → Postgres query or pgvector search + LLM answer
 |    └── tests/            # accuracy tests against your labeled samples
```

```
|
└── frontend/
    ├── package.json
    └── src/
        ├── App.jsx
        ├── api.js            # talks to FastAPI backend
        └── components/
            ├── MetricCard.jsx
            ├── CategoryChart.jsx
            ├── SentimentChart.jsx
            ├── ThemeList.jsx
            ├── FeedbackTable.jsx
            ├── WeeklySummary.jsx   # displays the narrative summary
            └── ChatWidget.jsx      # the chatbot box
```

InsightPulse— Final Tech Stack

Backend / Core

- Python — backend and AI pipeline language
- FastAPI — API layer connecting backend logic to the frontend
- pydantic — validates LLM JSON output against your schema, triggers retries on malformed responses

Database

- PostgreSQL — single database for structured feedback data
- pgvector — Postgres extension for storing embeddings and running similarity search in the same database (replaces ChromaDB)

AI / ML

- OpenAI GPT — classification, sentiment, urgency, theme extraction, and the narrative weekly summary
- Sentence-Transformers — converts feedback text into embeddings locally, free, for the chatbot's semantic search

Data Processing

- pandas — cleaning raw feedback, weekly aggregation (theme counts, sentiment trend)

Frontend

- React — dashboard UI
- Vite — dev server/build tool for React
- Chart.js — category and sentiment charts

Config / Security

- python-dotenv — loads API keys and DB credentials from .env, keeps secrets out of code

That's the complete stack end to end — nine tools, one language (Python) on the backend, one on the frontend (JS/React).